

Dynamic Context-Aware Tries for Knowledge Graph-Constrained Large Language Model Generation

Bernard Kirk Adjanor Katamanso¹, Erica Amonor¹, Joseph Osei Nyarko¹, Jessica Afua Etonam Nsafuah¹

¹University of Mines and Technology, Tarkwa, Ghana

Abstract

Knowledge Graph constrained decoding eliminates hallucination in large language model (LLM) reasoning by restricting output to verified paths. The leading approach, Graph Constrained Reasoning (GCR), achieves zero hallucination on Knowledge Graph Question Answering (KGQA) benchmarks by encoding all structurally valid paths in a static trie. The constraint is absolute but indifferent to the question: a three-hop query with twenty out-degree relations admits roughly 24,000 paths, and the LLM must navigate among them using the same internal knowledge that causes hallucination. We introduce Dynamic Context-Aware Tries (DCA-Trie), a dynamic constrained decoding framework that prunes the valid path set during generation using a TypeOracle, a lightweight symbolic module that checks two ontology gates per candidate triple. On WebQSP, DCA-Trie removes 85.5% of semantically irrelevant paths while maintaining a false negative rate below 5%. We further introduce the Semantic Irrelevance Ratio (SIR), a metric that separates constraint quality from answer accuracy, enabling independent evaluation of what the oracle filters versus what the LLM actually uses. SIR reveals a non-monotone relationship between constraint tightness and answer correctness: tighter constraints reduce the search space but do not improve Hits@1. We identify four mechanisms that drive this decoupling and show that the result holds across ablation conditions. DCA-Trie and SIR together provide the first framework for independently evaluating constraint quality and answer accuracy in knowledge graph-constrained decoding.

Keywords: knowledge graph question answering, constrained decoding, large language models, hallucination mitigation, trie-based decoding, semantic filtering, type oracle

1 Introduction

Large language models reason through generation by sampling each token from a distribution conditioned on all previous tokens (Brown et al. 2020; Touvron et al. 2023). This process produces fluent text, but fluency is not factual accuracy. LLMs hallucinate, generating confident claims that contradict source material or cannot be verified against any authoritative source (Ji et al. 2023; Huang et al. 2025). The problem worsens with scale where larger models hallucinate more convincingly, not less (Lin et al. 2022).

Despite this, many studies utilize knowledge graphs (KGs), which encapsulate extensive factual information in a structured format, to improve the reasoning abilities of LLMs (Pan et al. 2024; Luo et al. 2024). Because of the unstructured nature of LLMs, directly applying them to reason on KGs is challenging. Existing KG-enhanced LLM reasoning methods can be roughly categorized into three threads. Retrieval-based methods train specialized subgraph retrievers to extract question-focused subgraphs from the complete KG (Luo et al. 2024; Mavromatis and Karypis 2022). The concentrated subgraph is then fed to the LLM for answer prediction. These methods shift the burden of graph traversal to the retriever while the LLM merely selects from a pre-filtered set of candidates. Training the retriever requires annotated data, and the trained retriever may struggle with out-of-domain scenarios where the KG taxonomy differs from the training distribution (Li et al. 2025).

Agent-based methods take the opposite approach where the LLM itself interacts with the KG iteratively, executing operations such as neighbor exploration and path selection (Jiang et al. 2023a). These methods demand substantial computational resources and place high demands on the LLM’s instruction-following ability, which limits their applicability to small-scale models (Li et al. 2025).

Constrained decoding methods occupy a middle ground. Graph Constrained Reasoning (Luo et al. 2025), the leading framework, converts the KG into a trie-based index and constrains the LLM’s output to valid paths, achieving zero hallucination in the reasoning trace. Decoding on Graphs (Li et al. 2025) extends this by dynamically expanding the local trie as reasoning proceeds, adapting the constraint set to the partially generated chain. Both methods, however, share a common assumption: every structurally valid path should be admitted. Within L hops, GCR admits everything. At each step, DoG admits every topologically adjacent path. Neither filters for semantic relevance: whether a path is actually related to the question being asked.

The constraint is indifferent to what the question asks. In the worst case, a three-hop query with twenty out-degree relations and three question entities admits roughly 24,000 structurally valid paths. On WebQSP, where questions average 2.1 entities and 2 hops of reasoning depth, GCR still admits roughly 1,600 paths per question. Only one leads to the correct answer. The LLM must navigate among the rest using the same internal knowledge that produced hallucination in the first place. Tightening the constraint, filtering out irrelevant paths, seems like the obvious fix. We show that it is not.

We introduce DCA-Trie, a dynamic constrained decoding framework that conditions the valid token set on the knowledge graph, the input question, and the partially generated reasoning chain:

$$W_{\text{val}}^{\text{DCA}}(t) = f(G, E_q, q, y^{<t}) \quad (1)$$

The constraint set adapts at each decoding step. A lightweight symbolic module, the TypeOracle, checks two ontology gates per candidate triple. The range gate verifies that the tail entity’s type matches the relation’s range at every hop. The type gate verifies that the terminal entity’s type matches the inferred answer type at the final hop. Paths failing either gate are pruned before the LLM sees them.

Unlike embedding-based relevance scorers, the TypeOracle requires no neural encoder, no learned threshold, no per-dataset calibration, and no GPU memory for encoder inference. Gate evaluation is $O(1)$ per candidate triple, adding negligible overhead to the decoding process.

We introduce the Semantic Irrelevance Ratio (SIR), a metric that measures the fraction of structurally valid paths that are semantically irrelevant to the question. SIR separates constraint quality from answer accuracy: a system can have excellent SIR (tight constraints) and poor Hits@1 (wrong answer), or poor SIR (loose constraints) and excellent Hits@1 (correct answer). Without SIR, these two dimensions are entangled. A system with 91.6% Hits@1 and $\text{SIR} = 1.0$ (all paths admitted) looks identical to one with the same accuracy and $\text{SIR} = 0.1$ (90% filtered) if only Hits@1 is reported. SIR reveals which scenario holds.

On WebQSP, DCA-Trie reduces SIR from 1.0 to 0.145, an 85.5% removal of irrelevant paths, while maintaining a false negative rate below 5%. The TypeOracle that drives this filtering is purely symbolic: two ontology gates, $O(1)$ per candidate triple, no neural encoder, no learned threshold.

SIR also reveals that constraint tightness and answer accuracy are independently controlled variables. Tighter constraints reduce the search space but do not improve Hits@1. Four mechanisms govern this relationship: gold-path exclusion (the TypeOracle occasionally removes the correct path), beam competition (fewer paths reduce diversity in beam search), type-inference noise (the regex-based question classifier has imperfect accuracy), and the precision-recall trade-off (tighter constraints improve precision but reduce recall). The result holds across ablation conditions.

The contributions of this work are:

- (1) We introduce the Semantic Irrelevance Ratio (SIR), a metric that separates constraint quality from answer accuracy, enabling independent evaluation of what constrained decoding filters versus what the LLM actually uses.
- (2) We propose DCA-Trie, a dynamic constrained decoding framework that conditions the valid token set on the question and partial output via a TypeOracle with two ontology gates, reducing SIR by 85.5% with negligible latency overhead.
- (3) We establish that constraint tightness and answer accuracy are independently controlled variables, and characterise the four mechanisms (gold-path exclusion, beam competition, type-inference noise, and the precision-recall trade-off) that govern their relationship.

The remainder of this paper is organized as follows. Section 2 reviews related work on hallucination mitigation, knowledge graph reasoning, and constrained decoding. Section 3 presents the DCA-Trie framework, including the TypeOracle, the two-gate mechanism, and the SIR metric. Section 4 provides theoretical analysis of correctness and search-space reduction. Section 5 describes the experimental design, and Section 6 reports results. Section 7 presents extended analysis including ablation and sensitivity studies. Section 8 discusses implications and limitations, and Section 9 concludes.

2 Related Work

This section organizes the literature around four competing approaches to the same problem: generating text that is both fluent and factually grounded. Each subsection identifies what the approach achieves and where it stops, building toward the specific gap that DCA-Trie addresses.

2.1 Hallucination Mitigation

The most widely deployed defense against hallucination is retrieval-augmented generation (RAG) (Lewis et al. 2020). A dense retriever selects passages by embedding similarity, and the LLM generates from the retrieved context. Extensions such as FLARE (Jiang et al. 2023b) trigger retrieval when next-token confidence drops, and IRCOT (Trivedi et al. 2023) interleaves retrieval with chain-of-thought steps to improve multi-hop accuracy. Self-RAG (Asai et al. 2024) adds reflection tokens that control when to retrieve and whether retrieved passages support the generated claims.

These methods share a structural limitation. The retriever determines what the LLM *sees*; it does not determine what the LLM is *permitted to generate*. Because the generation mechanism remains unchanged, any token in the vocabulary retains a positive probability at every step, including tokens that correspond to no verified fact. Structural faithfulness with probability one cannot be achieved under any architecture that modifies only the input context without modifying the token selection mechanism (Geng et al. 2023; Luo et al. 2025).

Chain-of-thought (CoT) prompting (Bosma et al. 2022) takes a different approach: rather than retrieving facts, it elicits intermediate reasoning steps that decompose complex questions into simpler sub-problems. Self-consistency (Wang et al. 2023) samples multiple independent CoT chains and selects the final answer by majority vote, reducing sensitivity to individual hallucinated chains. Tree-of-Thought (Cao et al. 2023) treats generation as search through a tree of partial reasoning sequences, using the LLM as both generator and evaluator for pruning.

All prompting-based methods modify the *input* to the LLM without modifying the *generation mechanism*. A CoT chain can be entirely fluent and internally coherent while containing fabricated intermediate steps. Self-consistency reduces this risk through redundancy, but a majority of independently hallucinated chains can still produce an incorrect answer. Tree-of-Thought relies on the model’s own parameters for pruning decisions, without external verification. True structural faithfulness requires that invalid tokens receive exactly zero probability, achievable only by altering the decoding algorithm itself (Geng et al. 2023; Willard and Louf 2023).

2.2 Knowledge Graph Reasoning

Knowledge graphs encode verified facts as directed, labelled triplets (e, r, e') (Bollacker et al. 2008). KGQA requires identifying the answer entity by traversing chains of these triplets. Benchmarks such as WebQSP (Yih et al. 2016) and Complex WebQuestions (Talmor and Berant 2018) evaluate multi-hop path traversal over Freebase, demanding adherence to graph topology.

Early KGQA methods relied on semantic parsing: transforming questions into logical queries (e.g., SPARQL) for execution over the KG (Lan et al. 2022). These approaches guarantee faithfulness because answers are produced by verified graph operations. The requirement of training on ground-truth logical forms, and the non-executability of model-generated queries due to syntax or semantic errors, limits their practical deployment (Lan et al. 2022).

Retrieval-based methods use LLMs to extract relevant subgraphs or paths from the KG. RoG (Luo et al. 2024) trains a reasoning path planner that retrieves candidate paths and generates natural language explanations. GNN-RAG (Mavromatis and Karypis 2022) combines graph neural

networks with language models for retrieval over structured data. These methods improve relevance by filtering the KG, but the retrieved context remains a soft signal: it influences generation without constraining it.

Agent-based methods treat the LLM as an reasoning agent that iteratively queries the KG. ToG (Sun et al. 2024) prompts the LLM to explore relevant facts hop-by-hop using beam search on the KG. StructGPT (Jiang et al. 2023a) interfaces the LLM with structured data through iterative retrieval. These approaches are flexible but depend on the LLM’s instruction-following ability and are slow due to multiple KG queries per question.

The shared limitation across retrieval-based and agent-based KG reasoning is that retrieved context is a soft influence. The retriever or agent selects what the LLM observes; the LLM retains the freedom to generate any token. Even with perfect retrieval, hallucination remains structurally possible.

2.3 Constrained Decoding

Constrained decoding restricts generation to a predefined valid set at each step by directly intervening in the decoding mechanism. Unlike prompting or retrieval, which modify the input to the LLM, constrained decoding alters the token selection process itself. Invalid tokens receive zero probability before sampling, making their generation structurally impossible.

Format-constrained methods. Grammar-constrained decoding (GCD) (Geng et al. 2023) constrains generation to strings conforming to a context-free grammar via an Earley parser oracle. Outlines (Willard and Louf 2023) reduces the computational cost with a finite-state machine (FSM) formulation, precomputing an index from automaton states to valid vocabulary subsets for amortised $O(1)$ lookup. Additional methods include Synchromesh (Poesia et al. 2022) for program synthesis, PICARD (Scholak et al. 2021) for schema-constrained SQL parsing, and NeuroLogic Decoding (Lu et al. 2021) for propositional logic constraints.

These methods enforce output *format* rather than *factual content*. A grammatically valid SQL query can reference non-existent tables. A well-formed entity name can be factually incorrect. The extension from format constraints to KG path constraints requires a constraint oracle that encodes the relational structure of the knowledge graph itself.

KG-constrained methods. Graph-Constrained Reasoning (GCR) (Luo et al. 2025) builds a KG-Trie: a prefix tree of all KG paths within L hops of the question entities, created by breadth-first search before generation. The trie is used as the constraint oracle during beam-search decoding. GCR achieves 92.6% Hits@1 on WebQSP with 100% structural faithfulness using a Llama-3.1-8B backbone. The trie is constructed once and never updated: the valid token set is fixed regardless of what has been generated.

Decoding on Graphs (DoG) (Li et al. 2025) addresses GCR’s static oracle through step-wise trie expansion. After locking in an entity, the constraint trie is expanded with all KG neighbors of that entity, making the oracle topologically dynamic. DoG achieves 91.0% Hits@1 on WebQSP with 100% structural faithfulness. However, the expansion rule admits all neighbors regardless of relevance to the question, and step-wise trie reconstruction introduces computational overhead.

RouterKGQA (Yuan et al. 2026) routes between a specialized KG-reasoning model and a general LLM based on question complexity, using SBERT embeddings for post-hoc answer filtering. RouterKGQA’s semantic scoring operates at the answer selection level rather than at the token constraint level: paths are generated first, then filtered afterward.

The gap. All three KG-constrained methods determine their constraint sets from graph structure and question entities alone. None conditions the valid token set on question semantics or the partially generated chain. The constraint set is indifferent to whether a path is actually

related to the question being asked. On multi-hop questions, this produces valid token sets containing thousands of structurally correct but semantically irrelevant paths, forcing the LLM to navigate among them using its own knowledge and reintroducing the hallucination risk that constrained decoding was designed to eliminate.

2.4 Comparison and Research Gap

Table 1 compares the approaches reviewed above along four dimensions: whether the method enforces structural faithfulness (the generation mechanism, not just the input), whether the constraint set adapts during decoding, whether semantic relevance enters the constraint formation, and the type of oracle used.

Table 1: Comparison of KG-grounded reasoning methods. **Dynamic** indicates whether the constraint set changes during decoding. **Semantic** indicates whether question meaning enters the constraint oracle. The oracle type shows what drives constraint formation.

Method	Structural	Dynamic	Semantic	Oracle Type
RAG	No	No	Soft	Retriever
CoT	No	No	No	None
Self-RAG	No	No	Soft	Reflection
GCD	Format	No	No	Grammar/FSM
Outlines	Format	No	No	FSM-index
GCR	Yes	No	No	Static KG-Trie
DoG	Yes	Partial	No	Topology
RouterKGQA	Prob.	No	Yes	SBERT filter
DCA-Trie	Yes	Yes	Yes	TypeOracle

The table reveals a structural division. Methods above GCR achieve either semantic awareness (RAG, RouterKGQA) or structural faithfulness (GCD, Outlines), but not both. GCR and DoG achieve structural faithfulness but lack semantic awareness: their constraint oracles are indifferent to question meaning. No existing method achieves all three: structural faithfulness, dynamic adaptation, and semantic conditioning at the constraint level.

DCA-Trie fills this gap. Its TypeOracle conditions the valid token set on the knowledge graph, the question semantics, and the partially generated chain. The constraint set adapts at every decoding step, and only semantically relevant paths are admitted. The result is a method that achieves structural faithfulness while reducing the search space to paths the LLM actually needs to consider.

3 From Static to Context-Aware Constraints

Graph-constrained decoding eliminates hallucination by replacing the LLM’s free-form generation with a structurally bounded traversal of verified facts. GCR (Luo et al. 2025) operationalises this through KG-Trie, a prefix tree that encodes all reasoning paths within L hops of the question entities. At each decoding step, the trie yields the valid next-token set in $O(1)$ time. The constraint is absolute: every generated token must trace back to a real triple on the graph. Hallucination in the reasoning path drops to zero.

Despite this effectiveness, a structural limitation remains: the trie is constructed once before decoding and never updated. Its content depends only on the graph structure and the question entities:

$$\mathcal{W}_{\text{val}}^{\text{GCR}}(t) = f(\mathcal{G}, \mathcal{E}_q) \quad \forall t \quad (2)$$

It does not depend on the question q or the partial reasoning chain $y^{<t}$, so for a three-hop question with average out-degree $d \approx 20$ and three question entities, GCR admits roughly $3 \times 20^3 = 24,000$ structurally valid paths. Only one leads to the correct answer. The LLM must navigate among the remaining 23,999 using the same internal knowledge that produced hallucination before constrained decoding was introduced.

To this end, we propose DCA-Trie, a dynamic constrained decoding framework that conditions the valid token set on the knowledge graph, the input question, and the partially generated reasoning chain:

$$\mathcal{W}_{\text{val}}^{\text{DCA}}(t) = f(\mathcal{G}, \mathcal{E}_q, q, y^{<t}) \quad (3)$$

where $y^{<t} = (y_1, \dots, y_{t-1})$ is the generated prefix at step t . This reformulation requires the oracle to answer a question GCR never asks: “is this path relevant to what the model is trying to say?”

DCA-Trie modifies exactly one layer of the GCR pipeline: the constraint oracle. Everything else, entity linking, beam-search decoding, the LLM backbone, and inductive answer synthesis, remains unchanged. This isolation is deliberate. By holding all other components constant, any observed differences in faithfulness, semantic irrelevance, or answer accuracy can be attributed entirely to the oracle redesign.

3.1 The TypeOracle: Two Ontology Gates

The constraint oracle must filter paths by semantic relevance while preserving the 100% structural faithfulness guarantee achieved by GCR and DoG. We considered three approaches before arriving at the final design.

The first used cosine similarity between `all-MiniLM-L6-v2` embeddings of the path and the question, admitting paths above a threshold τ . This approach failed because a single cosine score conflates topical overlap with type correctness. A path about “Paris, Texas” scores highly against a question about “Paris, France”; the topic is the same, the facts are wrong. The threshold τ was also sensitive to the dataset: a value that worked on WebQSP did not transfer to ComplexWebQuestions.

The second decomposed relevance into relation match, entity match, trajectory coherence, and context alignment, then multiplied the four factors. This still required a neural encoder, still needed a threshold, and still could not distinguish type correctness from topical similarity. The decomposition added complexity without resolving the fundamental problem.

The third, which we adopt, uses two symbolic gates derived from the Freebase ontology. No neural encoder. No learned threshold. No per-dataset calibration. Gate evaluation is $O(1)$ per candidate triple. We call this module the TypeOracle.

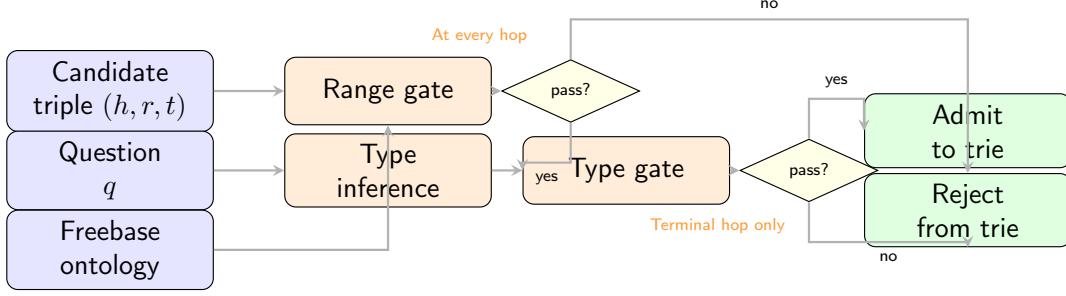


Figure 1: TypeOracle architecture. A candidate triple (h, r, t) passes through two sequential gates. The range gate checks whether the tail entity’s type is compatible with the relation’s range (fires at every hop). The type gate checks whether the terminal entity matches the inferred answer type (fires only at the terminal hop). Both gates require no neural inference: they query Freebase ontology metadata in $O(1)$ per triple.

3.1.1 Gate 1: Range Gate (every hop)

The range gate checks whether the tail entity’s type is compatible with the relation’s expected range:

$$\text{range_gate}(r, e') = \mathbf{1}[\text{types}(e') \cap \text{range}(r) \neq \emptyset] \quad (4)$$

If the relation is `ex_president` (range: $\{\text{Person}\}$), the candidate entity must be a Person. A Country fails. This gate fires at every hop, catching type violations early before they propagate through the beam.

3.1.2 Gate 2: Type Gate (terminal hop only)

The type gate checks whether the terminal entity’s type matches the inferred answer type:

$$\text{type_gate}(e', q) = \mathbf{1}[\text{types}(e') \cap \mathcal{T}(q) \neq \emptyset] \quad (5)$$

where $\mathcal{T}(q)$ is the set of expected answer types inferred from the question. A question beginning with “who” infers $\{\text{Person}\}$; “where” infers $\{\text{Location}\}$. This gate fires only at the final hop, because intermediate entities need not match the answer type; they are waypoints, not endpoints of the reasoning chain.

3.1.3 Why the range gate fires at every hop but the type gate only at the end

Consider: “Who is the spouse of the ex-president of USA?” The question entity is USA. At hop 1, the LLM generates a relation (`ex_president`) and a tail entity (a specific ex-president). At hop 2, it generates another relation (`spouse_of`) and a final entity (the spouse). The range gate must fire at both hops: at hop 1 to verify that the ex-president is a Person, at hop 2 to verify that the spouse is a Person. But the type gate must fire only at hop 2, because the intermediate entity (USA, a Country) is not of the answer type (Person). Applying the type gate at hop 1 would reject USA, breaking the chain. The range gate catches type violations at every step; the type gate enforces the answer constraint only where it matters, at the end.

3.1.4 Type Inference and Unknown Schema

Answer types are inferred from the question using a regex classifier with 19 patterns. “Who” maps to Person, “where” to Location, “when” to Date/Time. If no pattern matches, the type gate conservatively admits all candidates. When the TypeOracle lacks schema information for